This document explains Jednostava CPU instruction set arhitecture and instruction execution cycle

Created on 29.10.2026. by Ivan Jonjic (IJPantic on github)
Updated on 6.10.2026. by Ivan Jonjic (IJPantic on github)

Instruction parts (From LSB to MSB):

1. OP - Operation, 4-bit (ALU's operation, more on ALU operations later)

2. M - Mode, 1-bit (ALU's operation mode) -> {0: Arithmetic mode), 1: Logic mode}

3. CIN - Carry in, 1-bit (ALU's carry in) -> {0: Carry in), 1: No carry in}

4. ARGS - Argument Source, 2-bit (Argument of a data source)

5. SRC - Source, 3-bit (Instruction's data source)

6. ARGO - Argument opcode, 2-bit (Argument of a opcode)

7. OPC - Opcode, 3-bit (Instruction's opcode)

Opcodes table:

0 - PF (Pointer File), ARGO (Selects which pointer will be written to, note that write will occur only if jump condition is true) -> {0: PNT0 (PCI), 1: PNT1 (API), 2: PNT2 (PC), 3: PNT3 (AP)}

1 - A (Accumulator), no ARGO (X)

2 - RF (Register File), ARGO (Selects which register will be written to) -> {0: R0 (B), 1: R0 (C), 2: PNT2 (D), 3: PNT3 (RESR)}

3 - SF (Sector File), ARGO (Selects which sector will be written to) -> {0: SCT0 (sct PCI), 1: SCT1 (sct API), 2: SCT2 (sct PC), 3: SCT3 (sct AP)}

4 - CMP (Compare), ARGO (Selects condition which will be checked) -> {0: NEG, 1: CO, 2: EQU, 3: INTM}

5 - DVR (Direct Value Register), ARGO (Selectes which 8-bit part of register will be written to) -> {bit 0: Low part, bit 1: High part}

6 - ADRD (Address Device), no ARGO (X)

7 - INTC (Interrupt Control), ARGO (Selectes which action will be done) -> {bit 0: Enables interrupts, bit 1: Sets interrupt} (Note that setting interrupt has higher priority than enabling interrupts)

Sources table:

0 - PF (Pointer File), ARGO (Selects which pointer will be read from) -> {0: PNT0 (PCI), 1: PNT1 (API), 2: PNT2 (PC), 3: PNT3 (AP)}

1 - No SRC (X), no ARGO (X)

2 - RF (Register File), ARGO (Selects which register will be read from) -> {0: R0 (B), 1: R0 (C), 2: PNT2 (D), 3: PNT3 (RESR)}

3 - SF (Sector File), ARGO (Selects which sector will be read from) -> {0: SCT0 (sct PCI), 1: SCT1 (sct API), 2: SCT2 (sct PC), 3: SCT3 (sct AP)}

4 - No SRC (X), no ARGO (X)

5 - DVR (Direct Value Register), no ARGO (X)

6 - ADRD (Address Device), no ARGO (X)

7 - INTB (Interrupt Bus), no ARGO (X)

Conditions table:

0 - NEG (Negative in 2KK complement)

1 - CO (Carry out, a+b>1)

2 - EQU (Equal, but it tehnically just checks if all bits are high, with that in mind it can be used to check various other conditions)

3 - INTM (Interrupt mode, checks if CPU is not in interrupt mode)

Artihmetic operations table:

0 - A+CIN!

1 - A||B+CIN!

2 - A||(!B)+CIN

3 - -1+CIN

4 - A+(A&&(!B))+CIN

5 - (A||B)+(A&&(!B))+CIN

6 - A-B-1+CIN

7 - (A&&(!B))-1+CIN

8 - A+(A&&B)+CIN

9 - A+B+CIN

10 - (A||(!B))+(A&&B)+CIN

11 - (A&&B)-1+CIN

12 - A+A+CIN

13 - (A||B)+A+CIN

14 - (A||(!B))+A+CIN

15 - A-1+CIN

Logic operations table:

0 - !A

1 - !(A||B)

2 - (!A)&&B

3 - LOW

4 - !(A&&B)

5 - !B

6 - A^B

7 - A&&(!B)

8 - (!A)||B

9 - !(A^B)

10 - B

11 - A&&B

12 - HIGH

13 - A||(!B)

14 - A||B

15 - A

Instruction execution cycle stages and actions:

INPUT_M - Allows data into CPU data bus

IR_OD - Disables instruction register output (So it can executes special counter increment instruction)

FETCH - Fetches next instruction

SRC_R - Releases instruction's source onto data bus

INPUT_GR - Inputs gate register (Value stored within is temporary, it acts as a 'gate' between in and out data bus within CPU

INPUT_PF - Inputs vaalue into counter pointer (Increments current program counter)

EXE - Exexcutes instruction's opcode

Execution cycle (16-bit variant):

Instruction fetch:

step 0 - INPUT_M, IR_OD

step 1 - INPUT_M, IR_OD, FETCH

step 2 - INPUT_M, IR_OD

Counter increment:

step 3 - SRC_R, IR_OD, INPUT_GR

step 4 - SRC_R, IR_OD, INPUT_PF

Instruction execution:

step 5 - SRC_R, INPUT_GR

step 6 - SRC_R, EXE

step 7 - SRC_R,

Instruction execution cycle steps(8-bit variant):

Low instruction part fetch:

step 0 - INPUT_M, IR_OD

step 1 - INPUT_M, IR_OD, FETCH

step 2 - INPUT_M, IR_OD

Counter increment:

step 3 - SRC_R, IR_OD, INPUT_GR

step 4 - SRC_R, IR_OD, INPUT_PF

Does nothing:

step 5 - SRC_R

step 6 - SRC_R

step 7 - SRC_R

High instruction part fetch:

step 8 - INPUT_M, IR_OD

```
        step 9  - INPUT_M, IR_OD, FETCH

        step 10 - INPUT_M, IR_OD

    Counter increment:

        step 11 - SRC_R, IR_OD, INPUT_GR

        step 12 - SRC_R, IR_OD, INPUT_PF

    Instruction execution:

        step 13 - SRC_R, INPUT_GR

        step 14 - SRC_R, EXE

        step 15 - SRC_R,
```